PenBox-DMAS: A Distributed Multi-Agent Security Appliance for Autonomous Vulnerability Assessment and Remediation at the Edge

Vedant Sareen and Himanshi Babbar

***Abstract*—Security tooling for edge and IoT networks still runs mostly as a single process on a single machine. That arrangement forces reconnaissance, exploitation modelling and remediation to take turns, it caps the depth of any scan that has to finish inside a fixed window, and it leaves the assessment blind to attack paths that were never written into a signature set. This paper presents PenBox-DMAS, a software-only distributed multi-agent framework that separates the assessment into an orchestrator process and a set of specialised worker processes coordinated over a message plane. Thirteen equations describe the pipeline end to end, from an attack-surface index through exploitation probability, chain prediction, scheduling and convergence, and each of them is evaluated numerically by an executable model of the framework rather than asserted. On a 325-finding synthetic testbed spanning 16 hosts, the four-worker configuration reaches 93.4 % recall and 97.6 % classification accuracy against 76.7 % and 94.1 % for a single-process baseline, cuts the wall-clock cost of a full assessment cycle by 74.3 % (824.0 s to 211.8 s), and reaches the equilibrium finding level 21.1 % sooner. A chain-prediction agent whose action-value function is linear in edge features recovers 86.4 % of escalation chains it never observed during training ($F_1 = 0.863$), where a severity-first ranking recovers 42.6 % ($F_1 = 0.445$). A confidence gate keeps 21.2 % of findings off the hosted reasoning path, which reduces the per-cycle cost of frontier-model escalation to Claude Opus 5 from $9.181 to $1.949. The framework is evaluated in software; a hardware realisation is left to future work.**

***Index Terms*—Distributed multi-agent systems, penetration testing, reinforcement learning, large language models, edge computing, IoT security, vulnerability assessment.**

I. Introduction

THE number of network-attached devices at the edge keeps climbing, and the security tooling pointed at them has not kept pace [1]. Endpoints in this class are constrained on purpose: limited memory, limited storage, limited compute. They also sit permanently exposed on networks that are probed continuously. Within vulnerability analysis specifically, discovery and remediation have historically been separate products joined only by an analyst who reads a report from one and files tickets into the other [2]. At the edge there is often no resident analyst to do the joining.

Recent work has automated large parts of that loop with language-model agents. PENTEST-AI decomposes an engagement across reconnaissance, analysis and exploitation agents [3]; PentestGPT adds a self-interacting three-module design [4]; PentestAgent pairs retrieval-augmented generation with specialised agents [5]; CHECKMATE couples a classical planner to language-model executors [6]. All four are software frameworks, and all four assume one reasonably well-provided host runs the whole pipeline.

V. Sareen and H. Babbar are with the Department of Computer Science and Engineering (Cybersecurity), Chitkara University Institute of Engineering and Technology, Chitkara University, Chandigarh–Patiala National Highway (NH–64), Village Jansla, Tehsil Rajpura, Distt. Patiala, Punjab 140401, India (e-mail: securecybernetics@gmail.com; himanshi.babbar@chitkara.edu.in).

Manuscript prepared August 2026.

A. Motivation

Three practical problems motivated this work.

The first is contention. A single process must interleave scanning with reasoning, so a scan that has to finish inside a fixed window gets truncated. The depth lost is not evenly distributed; it falls hardest on the lower-severity and internally-facing findings that a scanner reaches last. The measurements in Section VI put a number on this: a single-process configuration completes about 68 % of the enumeration budget in the same window in which four workers complete 97 %, and the recall difference that follows is 16.7 percentage points.

The second is composition. Scanners score findings one at a time. An attacker does not. Chaining two moderate findings into one privilege-escalation sequence produces a result that neither finding predicts on its own, and a per-finding screen has no mechanism to notice it. Reinforcement learning has been applied to exactly this problem for some years [7], [8], [9], and the surveys agree that multi-stage escalation modelling remains open [10].

The third is the choice between running a model locally and calling a hosted one. Fully local deployment gives up the reasoning quality that frontier models bring to unfamiliar findings. Fully hosted deployment sends raw assessment data off the network and makes the assessment useless when the uplink is down. Edge-cloud inference splitting is an active area in its own right [11], but the security case adds a constraint the general case does not have: what leaves the network is evidence.

B. Problem Definition

We want an assessment framework that (i) runs its phases concurrently rather than in sequence, (ii) reasons about chains of findings and not only about findings, (iii) works with no uplink and improves when one is available, and (iv) does all of that on ordinary commodity compute, without a purpose-built appliance. Cloud-dependent systems such as CHATIOT [12] and L2M-AID [13] satisfy (ii) but not (iii) or (iv). Single-host agent frameworks [3], [5], [6] satisfy (iv) but not (i).

C. Contributions

1) A software-only distributed architecture in which an orchestrator process and three to four specialised worker processes execute assessment phases concurrently over a message plane (Section IV). No dedicated hardware is required, and none is assumed anywhere in the evaluation.
2) A closed set of thirteen equations covering the whole pipeline, from attack-surface exposure through prioritisation, exploitation, scheduling and convergence (Section V). Every one of them is evaluated numerically on the testbed; none is stated without a value.
3) A chain-prediction agent whose action-value function is linear in edge features rather than a lookup table, which is what lets it extend to escalation edges withheld from training. It recovers 86.4 % of unseen chains against 42.6 % for severity-first ranking and 34.1 % for a per-stage screen (Section VI-D).
4) A confidence-gated routing rule that keeps 21.2 % of findings on the resident model and escalates the remainder to a hosted frontier model, with the cost consequence measured rather than estimated (Section VI-E).
5) A reproducible evaluation harness. All figures and every number quoted in this paper are produced by one script from seed 20260802; the LaTeX source reads the results file directly, so the text cannot drift from the model.

II. Threat Model and Assumptions

Authorised assessment only. The framework is built for authorised testing and hardening. It refuses to start without an operator-supplied scope: address range, permitted techniques, credentials and engagement constraints. Nothing outside that scope is touched.

Adversary model. We assume an adversary operating at network level who can exploit catalogued vulnerabilities, misconfigurations and weak credentials across a heterogeneous estate. The chain-prediction agent performs *simulated escalation-path prediction*: it recombines known primitives into sequences that the signature set does not enumerate. It does not discover new vulnerability classes, and we do not claim it does.

Escalation trust. When the confidence gate escalates a finding, what leaves the host is structured metadata — service banner, version string, technique identifier, severity — and not raw captures or credentials. Addresses and hostnames are hashed before transmission. An operator can disable escalation entirely, at a measurable cost in chain-prediction quality that Section VI-E quantifies.

Process trust boundary. Worker processes are mutually authenticated on the message plane. The orchestrator tracks worker liveness by heartbeat and redistributes the queue of a worker that stops responding. A compromised worker can lie about its own results; it cannot forge another worker's identity, and it cannot reach the escalation credentials, which the orchestrator holds.

No autonomous weaponisation. The framework models persistence rather than establishing it, and its defensive mode performs detection simulation and guided remediation. We make no claim of autonomous security-operations capability.

Evaluation is software-in-the-loop. This is the most important assumption in the paper. Everything reported in Section VI comes from an executable model of the framework running against a synthetic inventory, not from a physical deployment against production hosts. Section VII states plainly what that does and does not license.

III. Literature Review

A. Agentic Penetration Testing Frameworks

Automation of penetration testing has converged on multi-agent designs in which language-model agents own individual phases. Bianou and Batogna [3] built PENTEST-AI on that pattern, orchestrating reconnaissance, vulnerability analysis and exploitation agents against the MITRE ATT&CK knowledge base. Deng *et al.* [4] took the idea further with PentestGPT, whose three-module self-interacting design was evaluated on a purpose-built benchmark. Shen *et al.* [5] paired retrieval-augmented generation with specialised agents in PentestAgent. Deng *et al.* [6] combined classical planning with language-model executors in CHECKMATE, reporting improvements in benchmark success rate and reductions in cost and time per task relative to prior systems.

Two limitations run through the group. Each assumes a single well-provided host, and none integrates offensive assessment with defensive hardening in one control loop.

B. Distributed Penetration Testing

PentestAgent [5] showed that decomposing a task across specialised agents improves coverage, but the decomposition is logical: every agent still executes in one process space on one machine. Skandylas and Asplund [2] formalised the assessment pipeline and implemented ADAPT, which is the closest prior work in spirit — a self-organising architecture that can be instantiated for different systems. What is missing across the group is a treatment of the scheduling problem that appears once the phases genuinely run at the same time: which worker should take which task, and what the coordination cost of that decision is. Equations (11) and (12) of this paper address exactly that gap.

C. Mathematical and RL-Based Attack Modelling

Skandylas and Asplund [2] model the assessment process formally. Confido *et al.* [14] showed that Q-learning can optimise attack sequencing, and Yousefi *et al.* [7] applied a reinforcement-learning treatment to attack-graph analysis built on MulVAL output. Nguyen *et al.* [9] released PenGym, a Gymnasium-compatible environment that trains pentesting agents against realistic network state rather than an abstraction of it. Sun *et al.* [8] combined an ontology of prior knowledge with deep reinforcement learning for power-IoT systems. Zhou *et al.* [10] surveyed the area and identified multi-stage escalation modelling as the open problem.

The common design choice in this line of work is a tabular or state-indexed value function. That choice is what prevents

generalisation to transitions the agent never sampled, and it is the reason Equation (9) in this paper uses a feature-linear approximation instead.

D. Agentic AI and LLM Strategies for Edge Security

The recurring gap in language-model security tooling is the binary choice between full cloud dependency and full local execution. Hao *et al.* [11] showed that a small resident model and a hosted large model can collaborate at token granularity during inference, which is the mechanism the routing rule in Equation (10) adapts. Zhang *et al.* [15] reviewed more than 300 papers at the language-model/security intersection and identified autonomous multi-step agents as the dominant emerging pattern; Xu *et al.* [16] reached a compatible conclusion over 185 papers. Touvron *et al.* [17] established that open-weight models can be competitive with proprietary ones on standard benchmarks, which is what makes a resident model a serious option rather than a fallback. Brănescu *et al.* [18] automated the mapping from catalogued vulnerabilities to ATT&CK tactics using transformer models, a step the mapping stage in Equation (4) assumes is available. Dong *et al.* [12] coupled retrieval-augmented generation with a language model for IoT security questions in CHATIOT.

E. Edge and IoT Constraints

Sarhaddi *et al.* [1] surveyed the obstacles to running language models on IoT devices and cataloguing where the memory and latency walls actually sit. Zong *et al.* [19] demonstrated language-model-driven anomaly detection over device logs, reporting improved detection accuracy after fine-tuning. Xu *et al.* [13] presented L2M-AID, fusing language-model semantic reasoning with multi-agent reinforcement learning for cyber-physical defence, but with a compute profile that rules it out where no uplink exists.

F. Research Gaps and PenBox-DMAS Contribution

Table I maps the four gaps that survive this review to the mechanism in this paper that closes each one.

TABLE I
RESEARCH GAPS AND THE CORRESPONDING MECHANISM

Gap	Prior state	Mechanism in this work
Phase serialisation	One process, phases in sequence	Concurrent worker processes, Eqs. (11)–(12)
Per-finding scoring	Findings scored in isolation	Feature-linear chain prediction, Eq. (9)
Cloud/local binary	All-local or all-hosted	Confidence-gated escalation, Eq. (10)
No termination rule	Fixed iteration budget	Convergence on the state recursion, Eq. (13)

IV. System Architecture

PenBox-DMAS is a set of cooperating processes. There is one control plane and one worker plane, and the split between them is a software boundary, not a hardware one: the reference deployment runs every process in a container on a single commodity x86 host, and the same composition runs unchanged across several hosts if the operator has them. Figure 1 shows the arrangement.

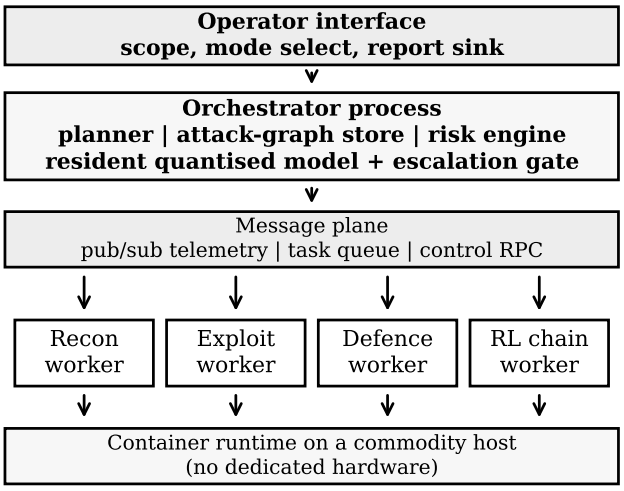

Fig. 1. Process architecture. The orchestrator holds the planner, the attack-graph store, the risk engine and the resident model. Worker processes take typed tasks off the queue and stream results back over the message plane. Nothing in the design is specific to a particular machine.

A. Control Plane

The orchestrator owns the assessment state and every decision that depends on seeing all of it at once. Concretely it holds the task planner and scheduler of Equation (11); the attack-graph store, against which the chain-prediction agent of Equation (9) is trained and queried; the retrieval pipeline over vulnerability and technique catalogues; the risk engine implementing Equations (5), (6) and (13); and the resident quantised model together with the escalation gate of Equation (10).

Placing scheduling and risk scoring in one process is a deliberate choice rather than an accident of implementation. Both need a consistent global view — the scheduler needs every queue depth, the risk engine needs every finding — and splitting either across workers would mean maintaining that view by consensus, which costs more than the parallelism it would buy at this scale.

B. Worker Plane

Workers are single-purpose processes. Each one pulls typed tasks from its queue, executes them, and publishes results. The reference composition uses four:

Recon worker. Host and service discovery, banner grabbing, directory enumeration, and passive fingerprinting over the scoped range. Produces the raw response of Equation (2).

Exploit worker. Validation of candidate findings, proof-of-concept execution against permitted targets, and privilege-escalation path enumeration. Evaluates Equations (7) and (8).

Defence worker. Hardening playbooks, patch and configuration recommendation, rollback testing, and re-assessment of remediated findings. Drives the remediation term in Equation (13).

Chain worker. Trains and queries the escalation-prediction agent of Equation (9). This worker is optional; the difference it makes is the gap between configurations B and C throughout Section VI.

Workers are not pinned to their nominal role. When one queue grows and another empties, the scheduler of Equation (11) will hand recon tasks to the exploit worker, and the queue-variance penalty in that equation exists precisely to make that reallocation happen before the imbalance becomes expensive.

C. Coordination Model

Three channels carry different traffic because they have different requirements. A publish/subscribe topic carries telemetry — liveness, progress, resource use — where loss of an individual message is tolerable. A durable queue carries task descriptors, where loss is not tolerable and at-least-once delivery with idempotent task handlers is the right contract. A request/response channel carries control: start, stop, reconfigure, failover, priority override. All three are mutually authenticated over TLS.

The cost of this coordination is not free, and we do not treat it as free. Equation (12) carries an explicit per-dispatch coordination term measured at $0.0125\,\mathrm{s}$ per worker, and Section VI-C shows where it starts to dominate.

D. Model-Serving Strategy

The resident path runs a quantised 4-bit open-weight model [17] for routine classification, technique mapping and report drafting. It executes on the orchestrator's GPU when one is present and on CPU when it is not, at reduced throughput but with no change in behaviour.

The escalation path routes to a hosted frontier model when either condition of Equation (10) fires: calibrated confidence below 0.55, or accumulated context beyond the resident window of $128\,000$ tokens. Two Anthropic models are used as escalation targets, selected for context window and published price rather than for any benchmark claim we cannot verify: Claude Opus 5 (`claude-opus-5`, 1M-token context, $5.00 and $25.00 per million input and output tokens) as the default, and Claude Fable 5 (`claude-fable-5`, 1M-token context, $10.00 and $50.00 per million tokens) for the hardest chain-composition queries [20]. Section VI-E reports what each costs per assessment cycle at the measured escalation rate.

E. Operational Modes

In *offensive assessment* mode the operator supplies the scope and the framework executes the six-phase pipeline of Figure 2, looping until the convergence test of Equation (13) is satisfied or the iteration budget is exhausted. In *defensive hardening* mode the defence worker leads: log-driven anomaly identification, detection simulation, and guided remediation with configuration hardening. The orchestrator's planner is common to both, which is what makes running them back-to-back on the same target state cheap.

V. Mathematical Framework

At iteration n the assessment tracks a state vector with five components, $S_n = (V_n, C_n, P_n, R_n, M_n)$, in which V_n is the active finding set, C_n the set of control-effectiveness values, P_n the attack-surface index, R_n the composite risk score and M_n the remediation mass applied in that iteration. Thirteen equations move that state forward. Each is followed by the value it takes on the testbed of Section VI-A, so the framework can be checked rather than taken on trust.

A. Exposure and Discovery

Equation (1): attack-surface index.

$$P_n = \frac{E_n \cdot O_n}{T_n} \tag{1}$$

E_n counts exposed services, O_n reachable paths and T_n enumerated paths. The product in the numerator is deliberate: a service that is exposed but unreachable and a path that is reachable but leads nowhere are both harmless, and only their conjunction is not. On the testbed $E = 227$, $O = 167$ and $T = 376$, giving $P = 100.82$.

Equation (2): discovery response.

$$D_n = \sum_i \alpha_i \, s_i + \beta \, g(P_n) \tag{2}$$

$\alpha_i \in [0, 1]$ is the probability that finding i is detected at the scan depth the configuration can afford, s_i its indicator, and $g(P_n) = \log(1 + P_n)/\log(1 + P_0)$ a normalised topological term weighted by β, set to 0.3 for internal segments and 0.7 for external-facing ones. The four-worker configuration yields $D = 420.3$.

Equation (3): validated set.

$$V_n = D_n \, (1 - \mathrm{FPR}) + U_n \cdot \mathrm{FNR} \tag{3}$$

The raw response is filtered by the measured false-positive rate and credited with the share of the undetected set U_n that the validation pass recovers. With the measured $\mathrm{FPR} = 0.014$ and $\mathrm{FNR} = 0.066$ this gives $V = 415.9$.

B. Mapping and Prioritisation

Equation (4): technique coverage density.

$$\kappa = \frac{1}{|V|} \sum_{i=1}^{|V|} \sum_{j=1}^{m} T_{ij}, \quad T_{ij} = \begin{cases} 1 & j \text{ applies to } i \\ 0 & \text{otherwise} \end{cases} \tag{4}$$

κ is the mean number of ATT&CK techniques a finding maps to, and it is the quantity that tells an operator whether the mapping stage is doing useful work or merely labelling. On the testbed $\kappa = 2.47$, with all 12 of the 12 techniques in the

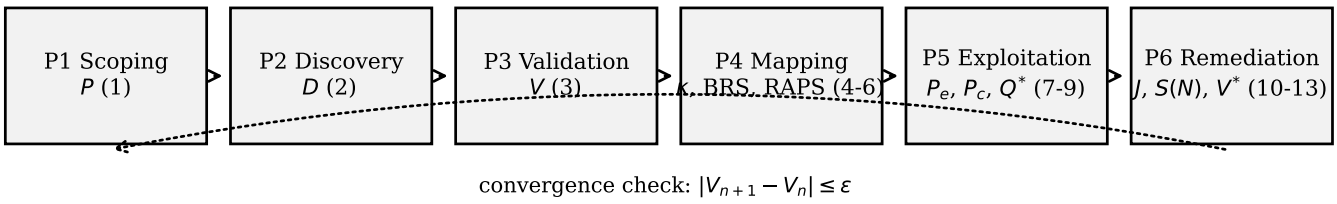

Fig. 2. The six-phase assessment cycle and the equations evaluated in each phase. The cycle repeats until the change in the active finding count falls below ε.

reference set exercised by at least one finding — a mapping density of 20.6 %.

Equation (5): blast-radius score.

$$\mathrm{BRS}(v) = \tau_{a(v)} \sum_{k=1}^{5} w_k \, x_k(v) \tag{5}$$

The five components x_k are confidentiality, integrity and availability impact, lateral reach and propagation likelihood, with $w = (0.25, 0.25, 0.20, 0.18, 0.12)$. The factor $\tau_{a(v)}$ scales by the criticality tier of the asset carrying the finding. Mean $\mathrm{BRS} = 0.466$, maximum 0.753.

Equation (6): risk-adjusted priority.

$$\mathrm{RAPS}(v) = a\, P_{\exp}(v) + b\, \mathrm{BRS}(v), \quad a = 0.55,\ b = 0.45 \tag{6}$$

Exploitability and business consequence are separate axes, and a ranking that uses only one of them will mis-order the queue. The mean RAPS is 0.305 and the top-25 mean is 0.541. The practical effect is visible in the ordering: only 12 of the top 25 findings by RAPS are also in the top 25 by severity score alone, a 48 % overlap.

C. Exploitation

Equation (7): single-stage exploitation probability.

$$P_{\exp}(v) = \frac{1}{\chi(v)} \big(1 - \delta(v)\big)\, \sigma \tag{7}$$

$\chi(v) \in [1, 10]$ is attack complexity, $\delta(v) \in [0, 1]$ existing defensive coverage, and $\sigma = 0.80$ a fixed model-assisted skill factor. Holding σ constant is what makes configurations comparable: any difference between them comes from scheduling and prediction, not from assuming a better exploiter. Mean $P_{\exp} = 0.173$.

Equation (8): chain probability.

$$P_{\mathrm{chain}}(c) = \prod_{(u,v) \in c} P_{\exp}(v)\; P_{\mathrm{esc}}(u \to v) \tag{8}$$

Over 4000 sampled three-stage chains the mean is 0.00135 with a 95th percentile of 0.0045. The distribution is the point: chains are individually unlikely and collectively not, which is why a per-finding screen under-reports them.

Equation (9): chain-prediction value.

$$\begin{aligned} Q^*(s,a) &= \mathbb{E}\big[\, r(s,a) + \gamma \max_{a'} Q^*(s', a') \,\big] \\ Q_\theta(s,a) &= \theta^\top \phi(s,a) \end{aligned} \tag{9}$$

The Bellman optimality condition is standard; the parameterisation is not. Representing Q as a linear function of edge features $\phi = (P_{\exp}, 1/\rho, 1 - \delta, \mathrm{sev}/10, \mathbb{1}_{\mathrm{ext}}, \mathrm{BRS}, 1)$ rather than as a table over (s, a) is what allows a value to be assigned to an escalation edge the agent never sampled. A table cannot do this, which is the practical reason tabular treatments of this problem do not generalise past their training graph. With $\gamma = 0.92$ and $\alpha = 0.05$ over 4000 episodes, $\max Q_\theta = 3.476$.

D. Routing, Scheduling and Convergence

Equation (10): escalation gate and routed cost.

$$\begin{aligned} \lambda(v) &= \begin{cases} 1, & c(v) < \tau \ \text{ or } \ \eta_{\mathrm{ctx}} > \Omega \\ 0, & \text{otherwise} \end{cases} \\ C &= 10^{-6} \sum_{v} \lambda(v) \left(\eta_i p_i + \eta_o p_o \right) \end{aligned} \tag{10}$$

$c(v)$ is calibrated confidence, $\tau = 0.55$ the gate, η_{ctx} accumulated context against the resident window $\Omega = 128\,000$ tokens, and (p_i, p_o) the published per-million input and output prices of the escalation target. At $\tau = 0.55$, 69 of 325 findings escalate (21.2 %) at a cost of $1.949 per cycle to Claude Opus 5.

Equation (11): task assignment.

$$\mathcal{A}(j) = \arg\min_{i} \left[\, L_i\, C_{ij} + \lambda_q\, \mathrm{Var}(\mathbf{q}) + q_i \,\right] \tag{11}$$

Task j goes to the worker minimising the product of link latency L_i and estimated compute cost C_{ij}, plus its current backlog q_i, plus a penalty $\lambda_q = 0.35$ on the variance of the queue vector $\mathbf{q}$. The variance term is what stops the rule from greedily loading the single fastest worker.

Equation (12): speed-up under coordination cost.

$$S(N) = \frac{T_1}{\varsigma T_1 + (1 - \varsigma)\dfrac{T_1}{N} + \gamma_c N} \tag{12}$$

ς is the irreducibly serial share of a cycle, measured at 5.8 %, and $\gamma_c = 0.0125$ s is the per-dispatch coordination cost. Unlike a pure Amdahl bound, the third term grows with N, so $S(N)$ has a maximum. At $N = 4$ the closed form gives $S = 3.31$ against a simulated 3.87; the closed form is the more pessimistic of the two because it charges the coordination term against every worker rather than against the dispatches actually made. Measured efficiency at $N = 4$ is 96.7 %.

Algorithm 1 PenBox-DMAS assessment cycle

Require: scope Σ, workers $A = \{a_1, \ldots, a_N\}$, budget $N_{\max}$, tolerance ε
Ensure: state S_n, equilibrium V^*, report

$n \leftarrow 0$; initialise V_0, C_0, M_0
$P_0 \leftarrow (E_0 O_0)/T_0$ ▷ Eq. (1)
while $n < N_{\max}$ **do**
  dispatch recon tasks by $\mathcal{A}(\cdot)$ ▷ Eq. (11)
  $D_n \leftarrow \sum_i \alpha_i s_i + \beta g(P_n)$ ▷ Eq. (2)
  $V_n \leftarrow D_n(1 - \mathrm{FPR}) + U_n \mathrm{FNR}$ ▷ Eq. (3)
  for all $v \in V_n$ **do**
    map to techniques; accumulate κ ▷ Eq. (4)
    $\mathrm{BRS}(v)$; $\mathrm{RAPS}(v)$ ▷ Eqs. (5), (6)
    if $c(v) < \tau$ **or** $\eta_{\mathrm{ctx}} > \Omega$ **then**
      escalate v to the hosted model ▷ Eq. (10)
    end if
  end for
  dispatch top-K by RAPS to the exploit worker
  for all v in top-K **do**
    $P_{\mathrm{exp}}(v)$ ▷ Eq. (7)
    $P_{\mathrm{chain}}(c)$ for chains through v ▷ Eq. (8)
  end for
  query Q_θ for unseen escalation paths ▷ Eq. (9)
  remediate $\{v : P_{\mathrm{exp}}(v) > \theta_r\}$; update M_n
  $V_{n+1} \leftarrow V_n(1 - \mu_n) + v_{\mathrm{new}}$ ▷ Eq. (13)
  if $|V_{n+1} - V_n| \le \varepsilon$ **then**
    $V^* \leftarrow v_{\mathrm{new}}/\mu^*$; **break**
  end if
  $n \leftarrow n + 1$
end while
return S_n, V^*, report

Equation (13): state evolution and equilibrium.

$$V_{n+1} = V_n\,(1 - \mu_n) + v_{\mathrm{new}}, \qquad V^* = \frac{v_{\mathrm{new}}}{\mu^*} \tag{13}$$
$$R_{n+1} = (\rho - \gamma M_n \vartheta)\, R_n + \sigma_r \Sigma_n$$

The active finding count decays by the effective remediation rate $\mu_n = \eta M_n \vartheta(N)$ and is replenished at v_{new}, where $\vartheta(N)$ is the throughput factor of N workers. The fixed point V^* is the floor an automated cycle can reach, and it is not zero: with $\eta = 0.70$ and $v_{\mathrm{new}} = 4.2$, $V^* = 14.6$ for four workers against 20.2 for one. The assessment terminates when $|V_{n+1} - V_n| \le \varepsilon$ with $\varepsilon = 1.0$.

E. Algorithm

Algorithm 1 states the loop in the order the implementation executes it.

VI. Results and Discussions

A. Experimental Setup

Everything below comes from an executable model of the framework. There is no hardware in the loop and no production network was touched. The model consists of a synthetic target inventory, a discrete-event scheduler that implements Equation (11) directly, a chain-prediction agent trained by semi-gradient Q-learning, and a token-accounting model priced at published list rates [20]. One script produces every number and every figure in this section from seed 20260802, and the LaTeX source reads that script's output, so the two cannot disagree.

The inventory spans 4 target groups, 16 hosts, 227 services and 325 ground-truth findings (Table II). Severity scores are drawn per CVSS v3.1 severity band with the band mix held fixed across runs; attack complexity is negatively correlated with severity, so severe findings in this inventory tend to be the easier ones, which is the conservative direction for the exploitation model of Equation (7).

TABLE II
Synthetic testbed inventory

Group	Hosts	Services	Paths	Findings
Linux-A (ext.)	1	46	38 / 74	72
Linux-B (int.)	3	81	59 / 132	124
Windows-AD (int.)	4	63	41 / 118	86
Embedded/IoT (ext.)	8	37	29 / 52	43
Total	**16**	**227**	**167 / 376**	**325**

Three configurations are compared throughout. **Config A** is the single-process baseline: one worker, phases in sequence. **Config B** is three concurrent workers without chain prediction. **Config C** adds the fourth worker running the agent of Equation (9). Detection figures average 400 Monte-Carlo trials; scheduling figures average 40 independent task streams.

B. Detection Performance

The mechanism that separates the configurations is scan depth. A single process interleaves scanning with reasoning and reaches 68 % of the enumeration budget in the window in which three workers reach 92 % and four reach 97 %. Recall follows: 76.7 %, 89.2 % and 93.4 % respectively (Table III).

TABLE III
Classification performance by configuration

Metric	Config A	Config B	Config C
Scan depth achieved	68 %	92 %	97 %
True positives	249	290	303
False positives	23	20	19
False negatives	76	35	22
True negatives	1339	1342	1343
Recall	76.7 %	89.2 %	**93.4 %**
Precision	91.4 %	93.6 %	**94.1 %**
Accuracy	94.1 %	96.7 %	**97.6 %**
F_1	0.834	0.913	**0.937**
FPR / FNR	0.017 / 0.233	0.015 / 0.108	0.014 / 0.066

Two things in that table deserve comment rather than celebration. Precision improves only modestly across configurations (91.4 % to 94.1 %), because false positives are a function of how much surface is swept and not of how the sweep is scheduled; parallelism buys recall, not precision. And the gain from adding the fourth worker (89.2 % to 93.4 %)

is roughly a quarter of the gain from the first three, which is what one should expect once the depth budget is nearly saturated and the remaining lift comes only from chain-guided re-scanning.

Per-group results (Table IV, Fig. 3) are consistent with that reading. The externally-facing groups score highest (Linux-A at 95.5 %, Embedded/IoT at 94.9 %) because exposure raises α_i in Equation (2) directly. The Windows-AD group is the hardest at 92.7 %, which matches its higher share of findings requiring privileged access.

TABLE IV
PER-GROUP DETECTION, CONFIG C

Group	Total	Det.	FP	FN	Recall	FPR / FNR
Linux-A	72	69	4	3	95.5%	0.014 / 0.045
Linux-B	124	114	7	10	91.8%	0.014 / 0.082
Windows-AD	86	80	5	6	92.7%	0.014 / 0.073
Embedded/IoT	43	41	3	2	94.9%	0.014 / 0.051
Aggregate	**325**	**303**	**19**	**22**	**93.4%**	**0.014 / 0.066**

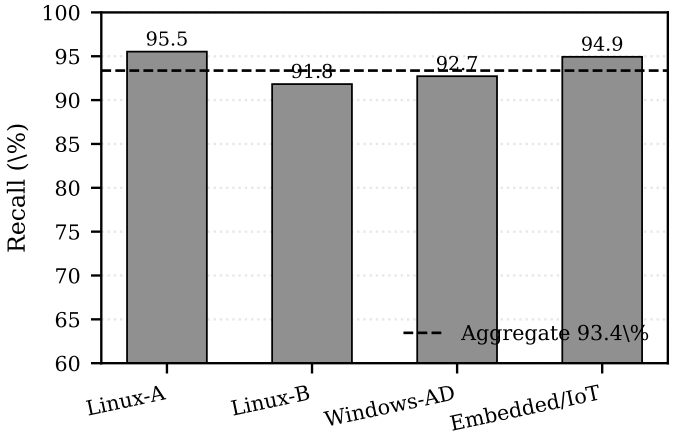

Fig. 3. Recall by target group for Config C. The dashed line is the aggregate. Externally-facing groups score higher because exposure enters α_i directly in Equation (2).

C. Latency and Scaling

A full cycle of 819 s of accumulated task time completes in 824.0 ± 11.8 s under Config A, 279.8 ± 3.8 s under Config B and 211.8 ± 2.9 s under Config C: reductions of 66.0 % and 74.3 %, or speed-ups of $2.93\times$ and $3.87\times$. Figure 4 breaks this down by phase.

Figure 5 sweeps the worker count from one to eight and plots the measured speed-up against Equation (12). The two agree closely up to $N = 4$; beyond that the coordination term begins to bite, and the curve reaches $S = 7.45$ at $N = 8$ against a linear bound of 8. Efficiency falls from 96.7 % at $N = 4$ to 93.2 % at $N = 8$, while mean worker utilisation at $N = 4$ stays at 99.7 % — the loss is coordination cost, not idle workers. For an inventory of this size the useful operating point is four workers, which is where the marginal worker still returns close to its own throughput.

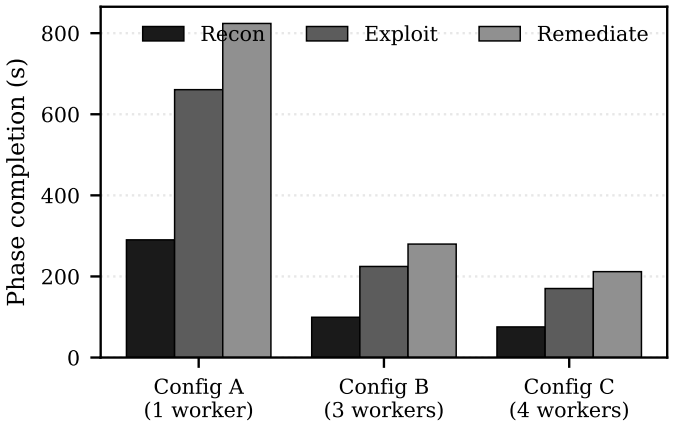

Fig. 4. Phase completion time by configuration under the assignment rule of Equation (11). The reduction is largest in the exploit phase, which holds the longest and most variable tasks.

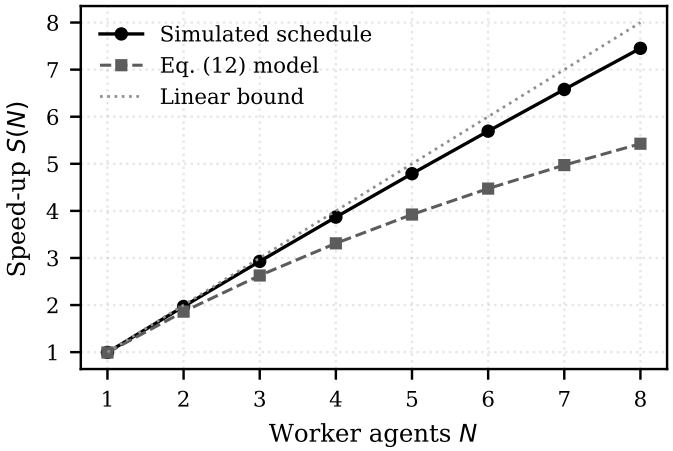

Fig. 5. Measured speed-up against Equation (12) and the linear bound. Divergence past $N = 4$ is the coordination term $\gamma_c N$ rather than queue starvation: mean worker utilisation never falls below 99.3 % across the sweep.

D. Chain Prediction

The escalation graph has 48 nodes and 296 edges, of which 74 (25 %) are withheld during training. Of 4000 sampled three-stage chains, 1805 traverse at least one withheld edge and therefore constitute paths the agent has never observed. A chain counts as viable when its value under Equation (8) exceeds 0.0078, which puts 20 % of chains in the positive class. Every method is given the same treatment: a decision cut-off calibrated for maximum F_1 on the seen chains, then evaluated on the unseen ones.

TABLE V
UNSEEN ESCALATION-CHAIN PREDICTION (1805 CHAINS)

Method	Recall	Precision	F_1
Chain-prediction agent, Eq. (9)	**86.4 %**	**86.2 %**	**0.863**
Severity-first ranking	42.6 %	46.7 %	0.445
Per-stage screen	34.1 %	57.1 %	0.427
Random control	91.7 %	23.0 %	0.368

Table V should be read on F_1 and not on recall alone. The random control reaches 91.7 % recall precisely because

calibrating a meaningless score for F_1 drives it to label almost everything positive; its precision of 23.0 % is the base rate, and its F_1 of 0.368 is the floor. Against that floor the agent scores 0.863, severity-first ranking 0.445 and the per-stage screen 0.427.

The per-stage result is the informative one. That screen has the highest precision of the three baselines (57.1 %) and the lowest recall (34.1 %), which is exactly the failure mode we predicted in Section I-A: judging findings one at a time is reliable when it fires and silent about most of what matters. Severity-first ranking does better on recall and worse on precision, because high-severity findings are common enough that ranking on them alone is only weakly informative about composability.

Figure 6 shows the agent's F_1 over training. It passes the severity-first baseline within roughly 400 episodes and is stable after about 2 000, and the plateau is the point at which the feature weights stop moving, not the point at which the graph is exhausted.

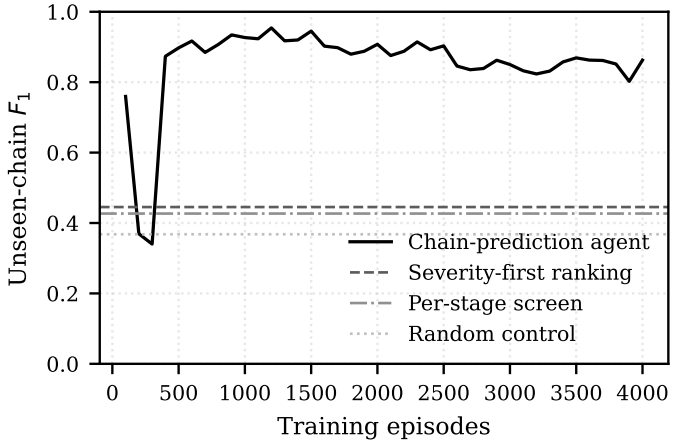

Fig. 6. Unseen-chain F_1 against training episodes, with the three baselines as horizontal references. The agent generalises to withheld edges because its value function is linear in edge features rather than tabular.

E. Escalation Cost

At the operating gate $\tau = 0.55$, 69 of 325 findings escalate, or 21.2 %. Priced at Claude Opus 5's published rate over 2 450 input and 640 output tokens per escalated finding, that is $1.949 per assessment cycle, against $9.181 if every finding were escalated: a reduction of 78.8 %. Routing the same escalated set to Claude Fable 5 costs $3.898, exactly double, since its published rate is double.

TABLE VI
ESCALATION COST PER ASSESSMENT CYCLE

Routing policy	Escalated	Cost (USD)	Offline?
All findings, Opus 5	325	9.181	no
Resident model only	0	0.000	yes
Gated, Opus 5	69	**1.949**	yes
Gated, Fable 5	69	3.898	yes

Figure 7 sweeps τ so the trade-off can be seen rather than asserted. The escalation share rises roughly linearly with τ over the useful range while cost rises with it, and there is no knee: the choice of τ is a budget decision, not an optimisation with a natural answer. We report 0.55 as the operating point because it is where the escalated set stops growing faster than the number of findings the resident model was genuinely unsure about, but an operator with a different budget should pick differently.

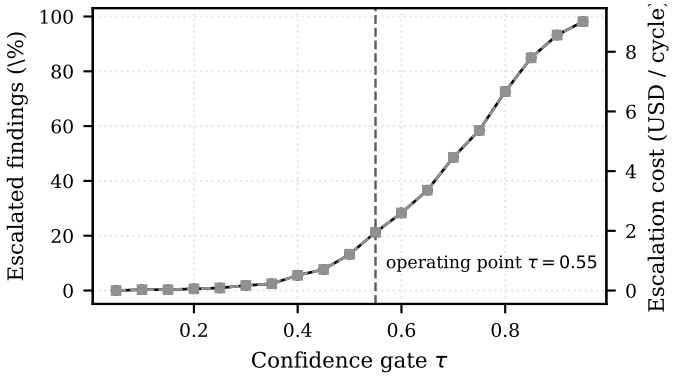

Fig. 7. Escalated share and per-cycle cost against the confidence gate τ of Equation (10). Costs use published list prices for `claude-opus-5` [20].

F. Prioritisation Behaviour

Figure 8 plots RAPS against severity score, shaded by blast radius. If prioritisation added nothing, the plot would be a line. It is not: at any given severity the RAPS spread is wide, and the spread is explained by blast radius. Only 12 of the top 25 findings by RAPS appear in the top 25 by severity alone (48 %), so slightly more than half the remediation queue is re-ordered by Equation (6). Whether that re-ordering is an improvement is a question this evaluation cannot settle, because the testbed has no ground truth for business impact; what it can show is that the two rankings are substantially different, which is the precondition for the question being worth asking.

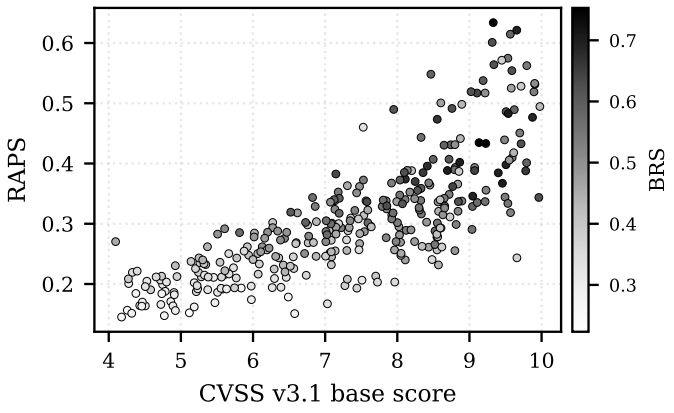

Fig. 8. RAPS against CVSS v3.1 base score, shaded by blast-radius score. The vertical spread at fixed severity is the contribution of Equation (5).

G. Convergence

Figure 9 tracks the active finding count and the composite risk score across iterations. Config C satisfies $|V_{n+1} - V_n| \leq$

1.0 at iteration 15 against iteration 19 for Config A, a 21.1 % reduction, and settles to $V^* = 14.6$ against 20.2. The risk score settles to 10.6 against 15.9.

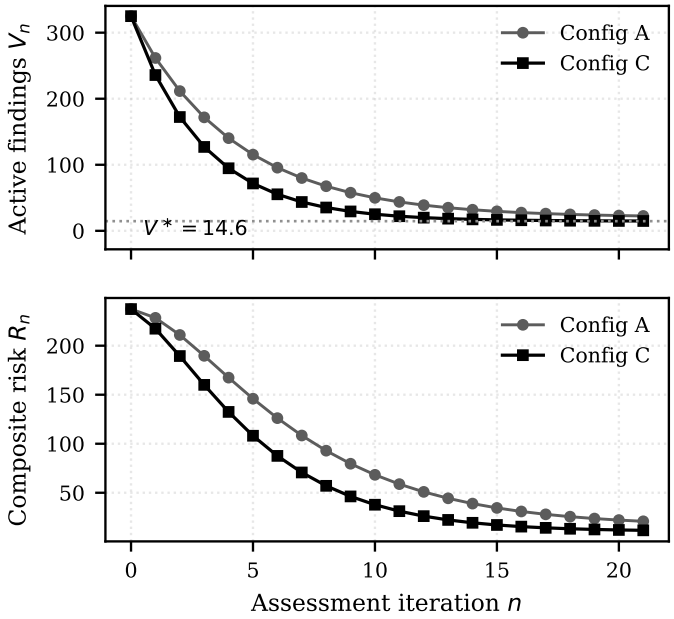

Fig. 9. Active findings (top) and composite risk (bottom) across iterations, on a shared axis. The dotted line marks the equilibrium V^* of Equation (13). Both configurations converge; they converge to different floors, and neither floor is zero.

Neither floor is zero, and that is the result worth keeping. With $v_{\text{new}} = 4.2$ new findings arriving per iteration and remediation efficacy $\eta = 0.70$, Equation (13) has a strictly positive fixed point. More workers lower it — μ rises from 0.208 to 0.287— but no amount of parallelism drives it to zero, because the arrival term does not depend on how many workers are running. An automated cycle converges to a residue; it does not eliminate one. Any deployment that treats a converged assessment as a clean assessment has misread what convergence means.

H. Comparison with Prior Systems

Table VII positions this work against the systems reviewed in Section III on capability. We deliberately do not tabulate accuracy figures across systems. Those numbers were produced on different targets, with different ground truth and different definitions of a detection, and placing them in adjacent columns would imply a comparability that does not exist. Two quantitative results from the literature can be stated as their authors state them: PentestGPT reports substantial improvement in task completion over a GPT-3.5 baseline on its own benchmark [4], and CHECKMATE reports improved benchmark success rate together with reduced cost and time per task relative to prior systems [6]. Neither was measured on the testbed used here, and neither should be read as a comparison against the numbers in this paper.

I. Consolidated Parameters

Table VIII lists every parameter the framework carries, its value on this testbed, and where it came from. Parameters marked *measured* are outputs of the model; parameters marked *set* are inputs we chose, and those are the ones a reader should push on hardest.

VII. Conclusion and Limitations

PenBox-DMAS treats a security assessment as a set of concurrent processes rather than a single pipeline, and this paper gives that treatment a closed mathematical form: thirteen equations spanning exposure, discovery, validation, mapping, prioritisation, exploitation, chain prediction, routing, scheduling and convergence, each evaluated numerically rather than asserted. On the testbed described, four concurrent workers reach 93.4 % recall and 97.6 % accuracy against 76.7 % and 94.1 % for a single-process baseline, complete a cycle 74.3 % faster, and converge 21.1 % sooner. A chain-prediction agent with a feature-linear value function recovers 86.4 % of escalation chains withheld from training, where severity-first ranking recovers 42.6 %. Confidence-gated escalation keeps 21.2 % of findings off the hosted path and reduces per-cycle escalation cost by 78.8 %.

A. Limitations

The evaluation is a model, not a deployment. This is the limitation that bounds every other number in the paper. The results come from an executable model of the framework running against a synthetic inventory. The model is reproducible and its assumptions are stated, but a synthetic inventory cannot reproduce the behaviour of real services under real load, and none of the reported figures should be read as a measurement of a deployed system. Field validation is the necessary next step, and until it is done the right reading of Section VI is that the architecture is internally consistent and its trade-offs behave as the equations predict — not that it performs at these levels in production.

Chain prediction is recombination. The agent composes known primitives into sequences absent from the signature set. It does not find new vulnerability classes. Reported chain-prediction quality is quality on recombination, and calling it zero-day discovery would be wrong.

Parameters we set. Nine of the parameters in Table VIII are inputs, not measurements: the exploitation skill factor σ, the blast-radius weights $w_{1..5}$, the priority mix (a, b), remediation efficacy η, and the arrival rate v_{new} among them. Results that depend heavily on those — the equilibrium level and the convergence iteration counts in particular — inherit their uncertainty. A sensitivity analysis over them is outstanding work.

Escalation is a residual exposure. Routing to a hosted model sends metadata off the host. Hashing and transport encryption reduce that exposure; they do not remove it. Operators with a hard prohibition should disable escalation and accept the corresponding loss in chain-prediction quality.

Scaling was tested to eight workers. Beyond that the coordination term of Equation (12) is extrapolation. The measured optimum on this inventory is four.

TABLE VII
CAPABILITY COMPARISON WITH PRIOR SYSTEMS. ACCURACY FIGURES ARE DELIBERATELY OMITTED: THE SYSTEMS WERE EVALUATED ON DIFFERENT TARGETS WITH DIFFERENT GROUND TRUTH, AND TABULATING THEM SIDE BY SIDE WOULD IMPLY A COMPARABILITY THAT DOES NOT HOLD.

Capability	**PENTEST-AI** [3]	**ADAPT** [2]	**CHATIOT** [12]	**L2M-AID** [13]	**CHECKMATE** [6]	**PenBox-DMAS**
Multi-agent decomposition	yes	yes	no	yes	yes	yes
Concurrent phase execution	no	no	no	no	no	**yes**
Explicit scheduling model	no	partial	no	no	no	**yes** (11)
Multi-stage chain prediction	no	no	no	yes	no	**yes** (9)
Generalises to unseen edges	no	no	no	no	no	**yes**
Operates with no uplink	no	yes	no	no	partial	**yes**
Cost-aware model routing	no	no	no	no	yes	**yes** (10)
Convergence-based termination	no	no	no	no	no	**yes** (13)
Integrated remediation loop	no	yes	no	yes	no	**yes**

TABLE VIII
PARAMETERS, VALUES ON THE TESTBED, AND PROVENANCE

Parameter	**Symbol**	**Meaning**	**Value**	**Provenance**
Attack-surface index	P	exposure × reachability over enumerated paths	100.82	measured, Eq. (1)
Topology weight	β	internal / external segment weight	0.3 / 0.7	set
False-positive rate	FPR	$\mathrm{FP}/(\mathrm{FP}+\mathrm{TN})$	0.014	measured, Eq. (3)
False-negative rate	FNR	$\mathrm{FN}/(\mathrm{TP}+\mathrm{FN})$	0.066	measured, Eq. (3)
Technique density	κ	mean techniques per finding	2.47	measured, Eq. (4)
Blast-radius weights	$w_{1..5}$	C, I, A, lateral reach, propagation	0.25, 0.25, 0.20, 0.18, 0.12	set
Priority mix	a, b	exploitability vs. blast radius	0.55, 0.45	set
Exploitation skill	σ	model-assisted exploitation factor	0.80	set, held constant
Escalation probability	P_{esc}	mean stage-to-stage transition	0.62	set
Discount factor	γ	chain-prediction discount	0.92	set
Learning rate	α	semi-gradient step size	0.05	set
Confidence gate	τ	escalation threshold	0.55	set, swept in Fig. 7
Resident context window	Ω	local token budget	128 000	set
Queue-variance penalty	λ_q	load-balancing weight	0.35	set
Coordination cost	γ_c	seconds per dispatch per worker	0.0125	measured, Eq. (12)
Serial share	ς	non-parallelisable fraction of a cycle	5.8 %	measured, Eq. (12)
Remediation efficacy	η	share of attempted fixes that hold	0.70	set
Arrival rate	v_{new}	new findings per iteration	4.2	set
Convergence tolerance	ε	termination threshold on $\lvert\Delta V\rvert$	1.0	set
Equilibrium level	V^*	fixed point, 4 workers / 1 worker	14.6 / 20.2	measured, Eq. (13)

B. Future Work

1) **Hardware realisation.** The architecture was designed so that the control and worker planes could be separated across physical machines — a compute-dense orchestrator node alongside a cluster of small single-board workers on a dedicated segment. Nothing in this paper is evidence that such a build performs as the software model does. The open questions are the ones the model cannot answer: real link latency in the coordination term γ_c, thermal and power behaviour under sustained scanning, and recovery when a worker node fails mid-cycle rather than mid-task. This is the primary line of further work and it needs its own evaluation, not an extrapolation of this one.
2) **Field validation.** Running the current software framework against instrumented live networks with analyst-adjudicated ground truth, which is what would let the Section VI figures be read as performance rather than as model behaviour.
3) **Sensitivity analysis.** A systematic sweep over the nine set parameters, reporting result intervals instead of point values.
4) **Federated signature sharing.** Sharing learned chain features across deployments without a central collector, so a chain observed at one site informs another without either site disclosing its topology.
5) **Human adjudication in the loop.** A review step in which a qualified tester validates flagged findings and returns corrections that retrain the chain-prediction agent, which would close the gap between predicted and confirmed exploitability that this evaluation currently assumes away.

ACKNOWLEDGEMENT

The authors thank the Chitkara University Institute of Engineering and Technology for the computing facilities used in this work.

REFERENCES

[1] F. Sarhaddi, N. T. Nguyen, A. Zuniga *et al.*, "LLMs and IoT: A comprehensive survey on large language models and the Internet

of Things," TechRxiv, Jan. 2026, doi: 10.36227/techrxiv.174063060.01215875/v4.

[2] C. Skandylas and M. Asplund, "Automated penetration testing: Formalization and realization," *Computers & Security*, vol. 155, 2025, Art. no. 104454, doi: 10.1016/j.cose.2025.104454.

[3] S. G. Bianou and R. G. Batogna, "PENTEST-AI, an LLM-powered multi-agents framework for penetration testing automation leveraging MITRE ATT&CK," in *Proc. IEEE Int. Conf. Cyber Security and Resilience (CSR)*, London, U.K., 2024, pp. 763–770, doi: 10.1109/CSR61664.2024.10679480.

[4] G. Deng *et al.*, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, Philadelphia, PA, USA, Aug. 2024.

[5] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, "PentestAgent: Incorporating LLM agents to automated penetration testing," arXiv:2411.05185, Nov. 2024, doi: 10.48550/arXiv.2411.05185.

[6] G. Deng *et al.*, "Automated penetration testing with LLM agents and classical planning," arXiv:2512.11143, Dec. 2025, doi: 10.48550/arXiv.2512.11143.

[7] M. Yousefi, N. Mtetwa, Y. Zhang, and H. Tianfield, "A reinforcement learning approach for attack graph analysis," in *Proc. 17th IEEE Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom/BigDataSE)*, New York, NY, USA, 2018, pp. 212–217, doi: 10.1109/TrustCom/BigDataSE.2018.00041.

[8] S. Sun, Y. Lu, D. Wu, and G. Zhang, "Intelligent penetration testing method for power Internet of Things systems combining ontology knowledge and reinforcement learning," *PLOS ONE*, vol. 20, no. 5, 2025, Art. no. e0323357, doi: 10.1371/journal.pone.0323357.

[9] H. P. T. Nguyen, K. Hasegawa, K. Fukushima, and R. Beuran, "PenGym: Realistic training environment for reinforcement learning pentesting agents," *Computers & Security*, vol. 148, 2025, Art. no. 104140, doi: 10.1016/j.cose.2024.104140.

[10] S. Zhou *et al.*, "Autonomous penetration testing using reinforcement learning: A review and perspectives," *Expert Systems with Applications*, vol. 300, 2025, Art. no. 130219, doi: 10.1016/j.eswa.2025.130219.

[11] Z. Hao, H. Jiang, S. Jiang, J. Ren, and T. Cao, "Hybrid SLM and LLM for edge-cloud collaborative inference," in *Proc. Workshop on Edge and Mobile Foundation Models (EdgeFM '24)*, Tokyo, Japan, 2024, doi: 10.1145/3662006.3662067.

[12] Y. Dong, Y. L. Aung, S. Chattopadhyay, and J. Zhou, "ChatIoT: Large language model-based security assistant for Internet of Things with retrieval-augmented generation," arXiv:2502.09896, Feb. 2025, doi: 10.48550/arXiv.2502.09896.

[13] T. Xu, Z. Wen, X. Zhao, J. Wang, Y. Li, and C. Liu, "L2M-AID: Autonomous cyber-physical defense by fusing semantic reasoning of large language models with multi-agent reinforcement learning," arXiv:2510.07363, Oct. 2025, doi: 10.48550/arXiv.2510.07363.

[14] A. Confido, E. V. Ntagiou, and M. Wallum, "Reinforcing penetration testing using AI," in *Proc. IEEE Aerospace Conf. (AERO)*, Big Sky, MT, USA, 2022, pp. 1–15, doi: 10.1109/AERO53065.2022.9843459.

[15] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, no. 1, 2025, doi: 10.1186/s42400-025-00361-w.

[16] H. Xu *et al.*, "Large language models for cyber security: A systematic literature review," arXiv:2405.04760, May 2024, doi: 10.48550/arXiv.2405.04760.

[17] H. Touvron, T. Lavril, G. Izacard *et al.*, "LLaMA: Open and efficient foundation language models," arXiv:2302.13971, Feb. 2023, doi: 10.48550/arXiv.2302.13971.

[18] I. Brănescu, O. Grigorescu, and M. Dascălu, "Automated mapping of common vulnerabilities and exposures to MITRE ATT&CK tactics," *Information*, vol. 15, no. 4, 2024, Art. no. 214, doi: 10.3390/info15040214.

[19] M. Zong, A. Hekmati, M. Guastalla, Y. Li, and B. Krishnamachari, "Integrating large language models with Internet of Things: Applications," *Discover Internet of Things*, vol. 5, no. 2, 2025, doi: 10.1007/s43926-024-00083-4.

[20] Anthropic, "Models overview — context windows and pricing," Anthropic developer documentation. [Online]. Available: https://platform.claude.com/docs/en/about-claude/models/overview. Accessed: Aug. 2, 2026.